SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2 – DEBUGGING & OPTIMISATION TASK

Course Code: CSA12	**Course Title:** Computer Architecture
CO Assessed: CO2	**Assessment:** ASSESSMENT TOOL2-DEBUGGING & OPTIMISATION TASK
Weightage: 20%	**Total Marks:** 25

CO2 Statement: Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)

Student Name: G NAVEEN KUMAR REDDY	**Reg.No.:** 192525288
Department: AIML	**Date:** 1/8/2026

ANNEXURE

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large positive values in 8-bit representation (for example +90 and +70). Debug the issue by analysing the binary computation and identify whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

2. A processor built around a ripple carry adder shows significant delay while executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit position. Suggest an optimised design using a carry look-ahead adder and explain how it reduces the critical path delay.

3. A multiplication unit implementing Booth's algorithm produces incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifting, and sign extension. Identify the likely source of the error and suggest corrections that ensure accurate signed multiplication.

4. An embedded system that uses the restoring division algorithm suffers from inefficiency because of repeated restoration steps, which increases execution time. Debug the algorithm's workflow, identify where unnecessary operations occur, and propose an optimised approach using the non-restoring division method, explaining how it improves performance.

5. A scientific computing application using IEEE-754 single-precision floating-point representation produces inaccurate results when adding a very small number to a very large one. Debug the issue by analysing exponent alignment, normalisation, and rounding, and suggest optimisation techniques such as switching to double precision or adjusting the summation algorithm.

Code of Conduct:

I, **G NAVEEN KUMAR REDDY** (192525288), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **G Naveen Kumar Reddy**

MARKS ALLOCATION

Question	Identification of Errors/Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors/Bugs	20%	Accurately identifies all errors with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root-cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides a completely corrected solution with accurate functionality and expected output	Provides a working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimisation methods that improve performance, memory usage and quality	Performs optimisation with noticeable improvements	Limited optimisation with minimal improvements	No optimisation applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Q1. Overflow Detection in Signed 2's Complement Addition

Problem: An 8-bit signed 2's-complement adder gives a wrong answer when the two operands +90 and +70 are added together.

1. Identifying the Error

An 8-bit signed 2's complement register can only hold values from −128 to +127. The true sum of the two operands is:

```
90 + 70 = 160
```

Since 160 lies outside the representable range, the register cannot hold the correct value, and the stored bit pattern will be misinterpreted as a negative number.

2. Working Out the Binary Computation

```
90  = 01011010

70  = 01000110

Sum = 10100000
```

Because the most significant bit of the result is 1, the hardware reads this pattern as a negative number:

```
10100000 → invert → 01011111 → add 1 → 01100000 = 96, so the pattern represents -96.
```

3. Confirming Overflow

Both inputs were positive, yet the stored result reads as negative (−96 instead of +160). This mismatch between the sign of the operands and the sign of the result is the classic signature of signed overflow.

Overflow status: YES

4. Corrected Approach

Overflow in 2's complement addition can be detected in two equivalent ways:

- Sign-bit rule: if two positive operands yield a negative sum, or two negative operands yield a positive sum, overflow has occurred.
- Carry-based rule: overflow = carry into the sign bit XOR carry out of the sign bit.

The arithmetic unit should raise an overflow flag rather than silently accepting the wrapped-around value. Where the application genuinely needs to represent 160, the fix is to widen the data path — for instance by using a 16-bit signed integer instead of 8-bit.

5. Verification / Testing

Operand A	Operand B	True Sum	8-bit Result	Overflow?
90	70	160	10100000	Yes
40	35	75	01001011	No
60	50	110	01101110	No
−90	−60	−150	01101010	Yes

Q2. Removing the Carry-Propagation Bottleneck in a Ripple Carry Adder

Problem: A ripple carry adder is too slow for a real-time system because each stage must wait for the carry produced by the stage before it.

1. Locating the Bottleneck

In a ripple carry adder, bit position i cannot compute its sum or carry-out until the carry-in from bit i−1 has settled. For an n-bit adder this creates a chain of n dependent stages, so the worst-case delay grows linearly with the word length.

2. Illustrating the Problem

```
11111111 + 00000001 = 100000000
```

Here a carry generated at bit 0 must ripple all the way through bit 7 before the final sum is valid — the worst possible case for this design.

3. Root-Cause Analysis Using Propagate/Generate Signals

For each bit position i, define:

```
Pi = Ai XOR Bi   (propagate)
```

```
Gi = Ai AND Bi   (generate)
```

The carry recurrence is $C_{i+1} = G_i + (P_i \cdot C_i)$, which still depends on the previous carry — this dependency is exactly what causes the delay.

4. Optimised Design: Carry Look-Ahead Adder

A carry look-ahead adder removes the chain dependency by expanding the recurrence so that every carry is written directly in terms of the P and G signals and the initial carry-in:

```
C1 = G0 + P0C0
```

```
C2 = G1 + P1G0 + P1P0C0
```

```
C3 = G2 + P2G1 + P2P1G0 + P2P1P0C0
```

Because none of these expressions wait on an intermediate carry, all carry bits can be produced by parallel logic gates rather than by sequential rippling.

5. Comparing the Two Designs

Aspect	Ripple Carry Adder	Carry Look-Ahead Adder
Carry computation	Sequential, stage by stage	Parallel, from P/G terms
Propagation delay	Grows linearly with bit-width	Nearly constant
Hardware complexity	Low	Higher (more gates)
Suitability for real-time use	Poor for wide operands	Well suited

6. Verification

For 255 + 1, both designs produce the identical arithmetic result 100000000. The difference is purely in how quickly that result becomes valid — the carry look-ahead design reaches a stable output in far fewer gate delays.

Q3. Correcting Booth's Algorithm for Signed Multiplication

Problem: A Booth multiplier produces wrong answers for some negative operands.

1. Worked Example

Multiply 6 × (−3) in 4-bit 2's complement.

```
M = 6 = 0110,  -M = 1010
```

```
Q = -3 = 1101,  A = 0000,  Q-1 = 0
```

2. Booth's Decision Table

Q0	Q−1	Operation
0	0	No arithmetic operation
0	1	A = A + M
1	0	A = A − M
1	1	No arithmetic operation

After the appropriate add/subtract, the combined register (A, Q, Q−1) is shifted arithmetically one place right. This is repeated once per bit of the multiplier — 4 times for a 4-bit operand.

3. Where the Bugs Usually Creep In

- Checking only Q0 instead of the pair (Q0, Q−1) — this skips the whole point of Booth encoding.
- Using a logical shift instead of an arithmetic shift, which destroys the sign bit (e.g. shifting 1010 must give 1101, not 0101).
- Failing to sign-extend A when it holds a negative partial result.
- Initialising Q−1 to something other than 0, or initialising A to a non-zero value.
- Running the wrong number of iterations (it must equal the operand width, not one more or one less).

4. Expected Result

6 × (−3) = −18. In 8-bit 2's complement:

```
18 = 00010010 → invert → 11101101 → add 1 → 11101110
```

So the correct product is $11101110_2 = -18$.

5. Corrected Procedure

- Represent both operands in 2's complement form.
- Initialise A = 0 and Q−1 = 0.
- At every step, examine the pair (Q0, Q−1) and apply the Booth decision table.
- Perform an arithmetic (sign-preserving) right shift across A, Q and Q−1 together.
- Repeat for exactly n cycles, where n is the operand width.
- Read the final product from the combined A and Q registers.

6. Verification

Multiplicand	Multiplier	Expected Product
6	−3	−18
−6	3	−18

6	3	18
−6	−3	18

Q4. Improving Restoring Division with the Non-Restoring Method

Problem: The restoring division algorithm wastes time because every unsuccessful subtraction must be undone with an extra addition.

1. How Restoring Division Works

At each step, the partial remainder and quotient are shifted left, the divisor is subtracted from the partial remainder, and the sign of the result is examined. If the result is negative, the divisor must be added back before the algorithm can move to the next bit — this add-back is the 'restoring' step.

2. Worked Example

Divide 22 by 4.

```
22 = 10110,  4 = 00100
```

22 ÷ 4 gives quotient Q = 5 and remainder R = 2.

3. Where the Inefficiency Comes From

Whenever the trial subtraction R − M is negative, restoring division performs a second operation, R = (R − M) + M, purely to undo the failed subtraction. In the worst case this doubles the number of arithmetic operations compared with what is actually needed.

4. Optimised Approach: Non-Restoring Division

Non-restoring division never undoes a subtraction. Instead, the sign of the current remainder decides whether the next step subtracts or adds the divisor:

```
If R ≥ 0:  R = 2R - M   (quotient bit = 1)
If R < 0:  R = 2R + M   (quotient bit = 0)
```

A final correction step adds the divisor back only once, at the very end, if the last remainder is negative — a single fix-up rather than a fix-up after every failed trial.

5. Comparing the Two Methods

Aspect	Restoring Division	Non-Restoring Division
Add-back on failure	Every time the trial subtraction fails	Never — decision instead flips to addition
Operations per cycle	Up to 2 (subtract + restore)	Exactly 1
Final correction needed	No	At most once, at the end
Overall speed	Slower	Faster

6. Verification

Check: divisor × quotient + remainder = 4 × 5 + 2 = 22, which matches the original dividend, confirming the result is correct.

Q5. Precision Loss in IEEE-754 Single-Precision Addition

Problem: Adding a very large floating-point number to a very small one produces an inaccurate result in single precision.

1. IEEE-754 Single-Precision Layout

A single-precision value uses 32 bits in total: 1 sign bit, 8 exponent bits (bias 127), and 23 fraction (mantissa) bits.

2. Illustrative Example

```
1.0        = 1.00000000000000000000000 × 2^0

2^-20      = 1.00000000000000000000000 × 2^-20
```

3. Why Exponent Alignment Causes Loss

Before the significands can be added, their exponents must be equalised. The smaller number's significand is therefore shifted right by 20 bit positions to line up with an exponent of 0. Because only 23 fraction bits are available, shifting by this many places pushes most or all of the smaller number's significant bits out of range.

As a result, 1.0 + 2^-20 rounds back to essentially 1.0 in single precision, even though the true mathematical sum is slightly larger.

4. Normalisation and Rounding

Once the significands are added, the sum is renormalised to the form 1.xxxxx × 2^E, and any bits that fall beyond the 23-bit fraction field are rounded off. When the two operands differ hugely in magnitude, the smaller operand contributes little or nothing to the final stored value.

5. Optimisation Techniques

- Use double precision (1 sign bit, 11 exponent bits, 52 fraction bits) when the application needs to preserve small quantities added to large ones.
- Group and sum values of similar magnitude together before combining them with much larger totals.
- Apply compensated summation techniques (e.g. Kahan summation) to track and recover rounding error across a long series of additions.
- Minimise the number of intermediate rounding steps in a computation chain.

6. Verification / Testing

Symptom	Root Cause	Fix
Exponents don't match before addition	Operands differ greatly in magnitude	Align exponents, accept the shift is unavoidable in principle
Mantissa bits dropped after shifting	Only 23 fraction bits available	Use a wider format (double precision)
Final result rounded incorrectly	Finite precision arithmetic	Use numerically stable summation algorithms
Small operand has no visible effect	Large magnitude gap between operands	Double precision or compensated summation